

⚠ Please do not reach out to reviewers directly; messaging them will not speed up your review.

Creating a Task

Learn how to create a new task from scratch using the task skeleton template.

Getting Started

To create a new task, you'll start with the task skeleton template and customize it for your specific task.

Step 1: Download the Correct Task Skeleton

Download the ZIP file of the correct task skeleton from the the [Slack channel's resources tab](#) located at the top of the channel.

There are 3 task skeletons:

- **"Regular" Task Skeleton:** Use for all non-UI-Building and non-Milestone tasks
- **UI Task Skeleton:** Use for all UI-Building subtype tasks
- **Milestone Task Skeleton:** Use for all tasks with milestones

Step 2: Extract and Rename

1. **Extract the ZIP file** to your desired location
2. **Rename the unzipped folder** to match your task name

Naming conventions:

- Use kebab-case (lowercase, hyphens)
- Be descriptive but concise
- Examples: `parse-json-logs`, `debug-python-import`, `configure-nginx-ssl`

Good task names:

-  `parse-json-logs`
-  `debug-python-import`
-  `configure-nginx-ssl`

Bad task names:

-  `task1`
-  `my-task`
-  `test`

Task Structure

After extracting and renaming, you'll have a folder structure like this:

Non-milestone tasks (Regular and UI skeletons):

 Copy

```
your-task-name/  
├─ instruction.md      # Task instructions (markdown)  
├─ task.toml          # Task configuration and metadata  
├─ environment/       # Environment definition folder  
│   ├─ Dockerfile     # OR docker-compose.yaml  
│   └─ [build files]  # Additional environment files  
├─ solution/          # Oracle solution  
│   └─ solve.sh       # Solution script + dependencies  
└─ tests/             # Test verification  
    ├─ test.sh        # Test execution script  
    └─ test_outputs.py # Python pytest assertions
```

Milestone tasks (Milestone skeleton):

 Copy

```
your-task-name/  
├─ task.toml          # Task config + one [[steps]] block  
├─ environment/       # Shared environment for all milestones
```

```

|   └─ Dockerfile                    # OR docker-compose.yaml
|   └─ [build files]
└─ steps/
    └─ milestone_1/
        └─ instruction.md            # Prompt for milestone 1 (inclu
        └─ tests/
            └─ test.sh                # Per-milestone test runner
            └─ test_m1.py            # Pytest assertions (TestMilest
        └─ solution/
            └─ solve.sh               # Wrapper – calls solve1.sh
            └─ solve1.sh             # Oracle solution scoped to mil
    └─ milestone_2/
        └─ ...                       # Same structure

```

Milestone tasks have **no** root-level `instruction.md`, `tests/`, `solution/`, or `milestone_x.md` files. See the [Milestones page](#) for full details.

Note: This structure follows the Harbor 2.0 task format. See [Task Components](#) for details on each file.

Next Steps

After downloading and renaming your task skeleton:

1. **Edit** `instruction.md` — Write clear, unambiguous task instructions
 - [Guide: Writing task instructions](#)
2. **Configure** `task.toml` — Set up metadata and configuration
 - [Guide: Task Components](#)
3. **Set up environment** — Configure Docker in the `environment/` folder
 - [Guide: Creating Docker environment](#)
4. **Write the solution** — Create `solution/solve.sh`
 - [Guide: Writing oracle solution](#)

5. **Create tests** — Write `tests/test.sh` and Python pytest files

- [Guide: Writing tests](#)

6. **Test locally** — Run the oracle agent to verify your task works

- [Guide: Oracle agent](#)

Related Resources

- [Running your task](#)
- [Writing oracle solution](#)
- [Writing tests](#)
- [Task Components](#)
- [Task Requirements](#)

PREVIOUS

[Example Tasks](#)

NEXT

[Running Your Task](#)